

OrderFlow PRD (2026-03-02)

Part 1: Basic Information

Title

OrderFlow - Backend API for LGTM Observability Stack Testing

Terminology

Term	Definition
Correlation ID	Unique identifier propagated across all services for a single request chain (used in distributed tracing)
Span	A unit of work in a distributed trace, representing a single operation
Trace	A collection of spans representing the full lifecycle of a request
SLI	Service Level Indicator - a quantitative measure of service behavior (e.g., request latency)
SLO	Service Level Objective - target value for an SLI (e.g., p99 latency < 500ms)
Hot Path	API endpoints with high traffic volume (product listing, health check)
Cold Path	API endpoints with low traffic but high computation (report generation)
Circuit Breaker	Pattern to prevent cascading failures when external services are down
Backpressure	Mechanism to slow down request processing when system is overloaded
Dead Letter Queue	Queue for failed messages that exceeded retry attempts
LGTM Stack	Loki (logging), Grafana (visualization), Tempo (tracing), Mimir (metrics) - a complete observability platform
RED Metrics	Rate, Errors, Duration - the three key metrics for monitoring request-driven services
OpenTelemetry (OTel)	Vendor-neutral observability framework for generating, collecting, and exporting telemetry data
W3C Trace Context	Standard HTTP headers (traceparent, tracestate) for distributed trace propagation
Chaos Endpoint	Admin API endpoint to dynamically inject failures and latency for testing observability

Project Information

Description

OrderFlow is a backend-only REST API built with NestJS, designed specifically for testing and learning the LGTM (Loki, Grafana, Tempo, Mimir) observability stack. The API simulates an e-commerce order management system with intentionally varied response characteristics - fast CRUD endpoints, medium-latency transactional endpoints, slow report/bulk endpoints, and error-prone external service simulations - to provide comprehensive scenarios for logging, monitoring, and distributed tracing.

Goals

1. Provide a realistic backend API with diverse performance characteristics (fast, medium, slow, error-prone endpoints) for LGTM stack testing
2. Enable comprehensive distributed tracing scenarios through cascading service calls (order → payment → inventory → notification)
3. Generate meaningful logs, metrics, and traces that exercise all components of the LGTM observability stack
4. Support dynamic failure injection via chaos endpoints for testing alerting and error monitoring capabilities

User Types

- **Customer:** End-user who browses products, places orders, views order history, and manages their own profile. Authenticated via JWT (email/password).
- **Admin:** System administrator who manages products, inventory, generates reports, views system health, and manages users. Accounts seeded via database migration (no public signup).

User Relationships

- Customer → Orders (1:N): A customer can have many orders
- Orders → Products (N:M via OrderItems): An order contains multiple products, a product can appear in multiple orders
- Admin manages all entities (independent): Admin has full CRUD access across all resources

Project Type

- Backend REST API - NestJS (TypeScript)
- Database - PostgreSQL (with TypeORM)
- Queue - BullMQ (Redis-backed)
- Observability - OpenTelemetry SDK → LGTM Stack

System Modules (Step-by-step Flows)

Module 1 - Product Catalog (FAST - target <50ms)

1. Customer or Admin sends `GET /products` with optional query params: `category` , `priceMin` , `priceMax` , `search` , `page` , `limit`
2. System queries PostgreSQL using indexed fields with pagination
3. System returns paginated product list with metadata (total count, page info)
4. **Observability:** Log request params + query time + result count; auto-span for DB query

Module 2 - Order Placement (MEDIUM - target 200-500ms)

1. Customer sends `POST /orders` with body: `{ items: [{ productId, quantity }] }`
2. System validates product existence (DB query ~20ms)
3. System checks inventory levels via `InventoryService` (service call ~50-100ms)
4. System calculates total pricing with tax (computation ~10ms)
5. System creates Order + OrderItems records in a transaction (DB write ~30ms)

6. System enqueues async payment processing job (queue push ~10ms)
7. System enqueues async notification job (queue push ~10ms)
8. System returns `201 Created` with order confirmation and tracking ID
9. **Observability:** Correlation ID assigned; span per sub-step with parent-child; log full order details + step durations

Module 3 - Payment Processing (SLOW - target 1-3s, async)

1. BullMQ worker picks up payment job from `payment-processing` queue
2. System calls simulated external payment gateway (configurable 500ms-2s latency)
3. On success (80%): Update order status to `paid`, enqueue inventory reservation job
4. On failure (20%): Retry up to 3 times with exponential backoff, then mark as `payment_failed` and push to dead letter queue
5. System enqueues payment confirmation/failure notification
6. **Observability:** Log payment attempt details + gateway response + retry count; trace full payment flow with external service span; metric: payment success/failure counter + gateway latency histogram

Module 4 - Report Generation (SLOW - target 2-5s)

1. Admin sends `POST /reports` with body: `{ type: "sales"|"orders"|"inventory", dateFrom, dateTo, format: "json"|"csv" }`
2. System validates date range and report type
3. System executes aggregation queries across orders, payments, and products tables (2-4s intentionally)
4. System formats result as JSON or CSV
5. System returns report data inline (small reports) or job ID for async retrieval (large reports)
6. **Observability:** Log query execution time + row counts + total generation time; individual query spans within report generation trace

Module 5 - Inventory Sync (MEDIUM/SLOW - target 300ms-2s)

1. Admin triggers `POST /inventory/sync` or system runs on CRON schedule
2. System calls simulated external warehouse API (variable latency 200ms-1.5s, 10% timeout rate)
3. System compares local stock levels with warehouse response
4. System updates local inventory records where discrepancies exist
5. System logs all discrepancies with product details
6. **Observability:** Log sync details + discrepancy count + external API response time; trace external service call span with status

Module 6 - Health & Metrics (FAST - target <20ms)

1. `GET /health` returns JSON: `{ status: "ok"|"degraded"|"down", checks: { database, redis, queue } }`
2. `GET /metrics` returns Prometheus exposition format with all collected metrics
3. Metrics include: request count, error rate, response time histograms, active connections, queue depth, business counters
4. **Observability:** Minimal logging (avoid health check noise); metrics endpoint scraped by Mimir/Prometheus

Module 7 - Chaos Engineering (FAST - admin only)

1. Admin sends `POST /chaos/config` with body specifying failure injection parameters
2. System updates in-memory chaos configuration (no DB persistence)
3. Configurable parameters: external service latency multiplier, failure rate percentages, artificial endpoint delays, enable/disable specific failure scenarios

4. GET /chaos/config returns current chaos configuration
5. POST /chaos/reset restores all defaults
6. **Observability:** Log all chaos config changes at WARN level with admin user context

3rd Party API List

Service	Type	Purpose
Simulated Payment Gateway	Mock (internal)	External payment API with configurable latency (500ms-2s) and failure rate (20% default) for tracing external calls
Simulated Warehouse API	Mock (internal)	External inventory service with variable response times (200ms-1.5s) and timeout rate (10%) for distributed trace testing

Note: Both integrations are fully simulated within the application. No real external services are called. Latency and failure rates are configurable via the /chaos endpoint.

Part 2: API Specification

Authentication

POST /auth/login

- **Request Body:** { email: string, password: string }
- **Response:** { accessToken: string, refreshToken: string, user: { id, name, email, role } }
- **Performance:** FAST (<50ms)
- **Auth:** Public

POST /auth/register

- **Request Body:** { email: string, password: string, name: string, phone?: string, address?: string }
- **Response:** { accessToken: string, user: { id, name, email, role } }
- **Performance:** FAST (<50ms)
- **Auth:** Public
- **Notes:** Only creates Customer accounts. Admin accounts are seeded via migration.

POST /auth/refresh

- **Request Body:** { refreshToken: string }
 - **Response:** { accessToken: string, refreshToken: string }
 - **Performance:** FAST (<50ms)
 - **Auth:** Public
-

Product Endpoints

GET /products

- **Query Params:** search?: string, category?: string, priceMin?: number, priceMax?: number, sortBy?: string, sortOrder?: "asc"|"desc", page?: number, limit?: number

- **Response:** { data: Product[], meta: { total, page, limit, totalPages } }
- **Performance:** FAST (<50ms)
- **Auth:** Public (no auth required)

GET /products/:id

- **Response:** { data: Product }
- **Performance:** FAST (<50ms)
- **Auth:** Public

POST /products

- **Request Body:** { name: string, description: string, price: number, category: string, sku: string, stockQuantity: number, imageUrl?: string }
- **Response:** { data: Product }
- **Performance:** FAST (<50ms)
- **Auth:** Admin only

PUT /products/:id

- **Request Body:** Partial Product fields
- **Response:** { data: Product }
- **Performance:** MEDIUM (200-500ms) - triggers inventory validation
- **Auth:** Admin only

DELETE /products/:id

- **Response:** { success: true }
- **Performance:** FAST (<50ms)
- **Auth:** Admin only

POST /products/bulk-import

- **Request Body:** { products: Product[] } or CSV file upload
- **Response:** { imported: number, failed: number, errors: [{ row, reason }] }
- **Performance:** SLOW (1-5s) - batch processing with progress logging
- **Auth:** Admin only
- **Observability Notes:** Generates many DB write spans; logs each batch progress; memory-intensive for large payloads

Order Endpoints

POST /orders

- **Request Body:** { items: [{ productId: string, quantity: number }], shippingAddress?: string }
- **Response:** { data: Order, trackingId: string }
- **Performance:** MEDIUM (200-500ms) - multi-step validation + inventory check + queue push
- **Auth:** Customer
- **Observability Notes:** Creates parent span with child spans for each sub-step; assigns correlation ID propagated to async payment/notification jobs

GET /orders

- **Query Params:** status?: string, dateFrom?: string, dateTo?: string, page?: number, limit?: number

- **Response:** { data: Order[], meta: { total, page, limit, totalPages } }
- **Performance:** MEDIUM (200-500ms) with filters, FAST (<50ms) simple listing
- **Auth:** Customer (own orders), Admin (all orders)

GET /orders/:id

- **Response:** { data: Order } (includes items, payment status, timeline)
- **Performance:** FAST (<50ms)
- **Auth:** Customer (own orders), Admin (all orders)

PATCH /orders/:id/status

- **Request Body:** { status: "confirmed"|"shipped"|"delivered"|"cancelled" }
 - **Response:** { data: Order }
 - **Performance:** FAST (<50ms)
 - **Auth:** Admin only
 - **Observability Notes:** Logs status transition; enqueues notification
-

Payment Endpoints (Async Processing)

POST /payments/process

- **Request Body:** { orderId: string }
- **Response:** { jobId: string, status: "queued" }
- **Performance:** FAST (<50ms) for enqueue; SLOW (1-3s) for async processing
- **Auth:** Internal / Admin trigger
- **Observability Notes:** Primary endpoint for testing distributed tracing with external service calls, retry patterns, and error scenarios. 20% simulated failure rate by default.

GET /payments/:orderId/status

- **Response:** { orderId: string, status: "pending"|"processing"|"paid"|"failed", attempts: number, lastError?: string }
 - **Performance:** FAST (<50ms)
 - **Auth:** Customer (own), Admin (all)
-

Inventory Endpoints

GET /inventory

- **Query Params:** lowStock?: boolean, category?: string, page?: number, limit?: number
- **Response:** { data: InventoryItem[], meta: { total, page, limit } }
- **Performance:** FAST (<50ms)
- **Auth:** Admin only

POST /inventory/sync

- **Response:** { synced: number, discrepancies: number, duration: number }
- **Performance:** SLOW (300ms-2s) - calls simulated warehouse API
- **Auth:** Admin only
- **Observability Notes:** Variable latency from external service; 10% timeout rate; logs all discrepancies at WARN level

PATCH /inventory/:productId

- **Request Body:** { stockQuantity: number }
 - **Response:** { data: InventoryItem }
 - **Performance:** FAST (<50ms)
 - **Auth:** Admin only
-

Report Endpoints

POST /reports

- **Request Body:** { type: "sales"|"orders"|"inventory", dateFrom: string, dateTo: string, format?: "json"|"csv" }
- **Response:** Report data (inline) or { jobId: string } for async retrieval
- **Performance:** SLOW (2-5s) - heavy aggregation queries
- **Auth:** Admin only
- **Observability Notes:** Primary endpoint for testing slow query detection; creates multiple DB query spans; logs execution plan and row counts

GET /reports/:jobId

- **Response:** { status: "processing"|"completed"|"failed", data?: ReportData, downloadUrl?: string }
 - **Performance:** FAST (<50ms) for status check
 - **Auth:** Admin only
-

Notification Endpoints

GET /notifications

- **Query Params:** read?: boolean, page?: number, limit?: number
- **Response:** { data: Notification[], meta: { total, unread, page, limit } }
- **Performance:** FAST (<50ms)
- **Auth:** Customer (own), Admin (all)

PATCH /notifications/:id/read

- **Response:** { data: Notification }
 - **Performance:** FAST (<50ms)
 - **Auth:** Customer (own)
-

User Management Endpoints (Admin)

GET /users

- **Query Params:** search?: string, role?: string, status?: string, page?: number, limit?: number
- **Response:** { data: User[], meta: { total, page, limit } }
- **Performance:** FAST (<50ms)
- **Auth:** Admin only

GET /users/:id

- **Response:** { data: User } (includes order count, last login)
- **Performance:** FAST (<50ms)
- **Auth:** Admin only

PATCH /users/:id

- **Request Body:** { status?: "active"|"inactive", role?: string }
 - **Response:** { data: User }
 - **Performance:** FAST (<50ms)
 - **Auth:** Admin only
-

Health & Observability Endpoints

GET /health

- **Response:** { status: "ok"|"degraded"|"down", timestamp: string, checks: { database: "ok"|"error", redis: "ok"|"error", queue: { depth: number, status: "ok"|"backlogged" } } }
- **Performance:** FAST (<20ms)
- **Auth:** Public

GET /health/ready

- **Response:** { ready: boolean } (Kubernetes readiness probe)
- **Performance:** FAST (<10ms)
- **Auth:** Public

GET /health/live

- **Response:** { alive: boolean } (Kubernetes liveness probe)
- **Performance:** FAST (<5ms)
- **Auth:** Public

GET /metrics

- **Response:** Prometheus exposition format text
 - **Performance:** FAST (<20ms)
 - **Auth:** Public (or internal network only)
 - **Notes:** Scraped by Mimir/Prometheus at configured interval
-

Chaos Engineering Endpoints (Admin)

GET /chaos/config

- **Response:** Current chaos configuration object
- **Performance:** FAST (<10ms)
- **Auth:** Admin only

POST /chaos/config

- **Request Body:**

```
{
  "paymentGateway": {
    "latencyMs": { "min": 500, "max": 2000 },
```

```

    "failureRate": 0.2,
    "timeoutRate": 0.1
  },
  "warehouseApi": {
    "latencyMs": { "min": 200, "max": 1500 },
    "timeoutRate": 0.1
  },
  "artificialDelays": {
    "enabled": false,
    "endpoints": {
      "GET /products": 0,
      "POST /orders": 0
    }
  },
  "queueSlowConsumer": {
    "enabled": false,
    "delayMs": 5000
  }
}

```

- **Response:** { config: ChaosConfig, appliedAt: string }
- **Performance:** FAST (<10ms)
- **Auth:** Admin only
- **Observability Notes:** All config changes logged at WARN level with admin context

POST /chaos/reset

- **Response:** { config: ChaosConfig, resetAt: string }
- **Performance:** FAST (<10ms)
- **Auth:** Admin only

Part 3: Observability Specification

This section defines the detailed observability requirements for testing with the LGTM stack.

3.1 Logging (Loki)

Log Format

All logs must be structured JSON with the following base fields:

```

{
  "timestamp": "2026-03-02T12:00:00.000Z",
  "level": "info",
  "message": "Order placed successfully",
  "service": "orderflow",
  "correlationId": "abc-123-def-456",
  "traceId": "4bf92f3577b34da6a3ce929d0e0e4736",
  "spanId": "00f067aa0ba902b7",
  "method": "POST",
  "path": "/orders",

```

```
"userId": "user-789",
"duration": 342,
"statusCode": 201
}
```

Log Levels & Usage

Level	Usage	Example
DEBUG	Detailed internal state, DB query params	"Querying products with filters: {category: 'electronics'}"
INFO	Normal operations, business events	"Order ORD-123 placed by user USR-456"
WARN	Slow queries (>100ms), retries, discrepancies	"DB query took 250ms for report generation"
ERROR	Failures, exceptions, timeouts	"Payment gateway timeout after 5000ms for order ORD-123"

Logging Requirements per Module

Module	Log Points
Product Catalog	Request params, query time, result count
Order Placement	Order details, each step duration, correlation ID, total processing time
Payment Processing	Attempt number, gateway response, retry count, final status, error details
Report Generation	Query execution time, row counts, total generation time
Inventory Sync	Sync details, discrepancy count, external API response time
Health Check	Minimal logging (only on degraded/down status)
Chaos Config	All config changes at WARN level with admin user context

Error Log Enrichment

Error logs must include:

- Full stack trace
- Sanitized request body (no passwords, tokens)
- User context (ID, role)
- Correlation ID for trace linking
- Service and module context

3.2 Metrics (Mimir/Prometheus)

RED Metrics (All Endpoints)

Metric Name	Type	Labels	Description
-------------	------	--------	-------------

http_requests_total	Counter	method, path, status_code, endpoint_category	Total HTTP requests
http_request_duration_seconds	Histogram	method, path, endpoint_category	Request duration with buckets: 0.01, 0.05, 0.1, 0.25, 0.5, 1, 2.5, 5, 10
http_request_errors_total	Counter	method, path, error_type	Total request errors

Business Metrics

Metric Name	Type	Labels	Description
orderflow_orders_placed_total	Counter	status	Total orders placed
orderflow_orders_value_total	Counter	currency	Total order value
orderflow_payments_processed_total	Counter	status, attempt	Total payment attempts
orderflow_payment_gateway_duration_seconds	Histogram	status	External payment gateway latency
orderflow_reports_generated_total	Counter	type, format	Reports generated
orderflow_report_generation_duration_seconds	Histogram	type	Report generation time
orderflow_inventory_sync_total	Counter	status	Inventory sync operations
orderflow_inventory_discrepancies_total	Counter	-	Stock discrepancies found

Infrastructure Metrics

Metric Name	Type	Labels	Description
orderflow_db_pool_active	Gauge	-	Active database connections
orderflow_db_pool_idle	Gauge	-	Idle database connections
orderflow_db_query_duration_seconds	Histogram	query_type	Database query duration
orderflow_queue_depth	Gauge	queue_name	Current queue depth
orderflow_queue_processing_duration_seconds	Histogram	queue_name	Queue job processing time

orderflow_queue_failed_total	Counter	queue_name, error_type	Failed queue jobs
------------------------------	---------	---------------------------	-------------------

Runtime Metrics

Metric Name	Type	Labels	Description
nodejs_eventloop_lag_seconds	Gauge	-	Node.js event loop lag
process_cpu_seconds_total	Counter	-	CPU usage
process_resident_memory_bytes	Gauge	-	Memory usage

3.3 Tracing (Tempo)

Trace Context Propagation

- Protocol: W3C Trace Context (`traceparent` and `tracestate` headers)
- All internal service calls must propagate trace context
- Queue jobs must carry trace context from the originating request

Span Naming Convention

- HTTP handlers: `HTTP {METHOD} {ROUTE}` (e.g., `HTTP POST /orders`)
- Database queries: `DB {OPERATION} {TABLE}` (e.g., `DB SELECT products`)
- Queue operations: `QUEUE {ACTION} {QUEUE_NAME}` (e.g., `QUEUE PUBLISH payment-processing`)
- External services: `EXT {SERVICE} {OPERATION}` (e.g., `EXT PaymentGateway charge`)

Span Attributes

Attribute	Type	Applied To	Description
user.id	string	All authenticated spans	User performing the action
user.role	string	All authenticated spans	User role (customer/admin)
order.id	string	Order-related spans	Order identifier
payment.id	string	Payment spans	Payment attempt identifier
endpoint.category	string	HTTP spans	fast/medium/slow/error-prone
db.system	string	DB spans	"postgresql"
db.statement	string	DB spans	SQL query (sanitized)
db.operation	string	DB spans	SELECT/INSERT/UPDATE/DELETE
messaging.system	string	Queue spans	"bullmq"
messaging.destination	string	Queue spans	Queue name
error	boolean	Error spans	Whether span represents an error
error.message	string	Error spans	Error description

Trace Scenarios for Testing

Scenario 1: Simple CRUD (1-2 spans)

```
HTTP GET /products/:id
└─ DB SELECT products (WHERE id = ?)
```

Scenario 2: Order Placement (5-7 spans)

```
HTTP POST /orders
├─ DB SELECT products (validate existence)
├─ InventoryService.checkStock
│   └─ DB SELECT inventory
├─ PricingService.calculate
├─ DB INSERT orders + order_items (transaction)
├─ QUEUE PUBLISH payment-processing
└─ QUEUE PUBLISH notifications
```

Scenario 3: Payment Processing (3-5 spans, async continuation)

```
QUEUE CONSUME payment-processing
├─ EXT PaymentGateway charge (500ms-2s)
├─ DB UPDATE orders (status)
├─ QUEUE PUBLISH inventory-reservation
└─ QUEUE PUBLISH notifications
```

Scenario 4: Report Generation (4-8 spans)

```
HTTP POST /reports
├─ DB SELECT orders (aggregation, 1-2s)
├─ DB SELECT payments (aggregation, 500ms-1s)
├─ DB SELECT products (join, 500ms)
└─ ReportFormatter.format
```

Scenario 5: Failed Payment with Retries (8-12 spans)

```
QUEUE CONSUME payment-processing
├─ EXT PaymentGateway charge → TIMEOUT (5s)
├─ [Retry 1] EXT PaymentGateway charge → ERROR (500)
├─ [Retry 2] EXT PaymentGateway charge → SUCCESS
├─ DB UPDATE orders (status = paid)
└─ QUEUE PUBLISH notifications
```

3.4 API Response Time Design

Endpoint	Method	Target Latency	Category	Testing Purpose
/health	GET	<20ms	Fast	Baseline, liveness/readiness
/health/ready	GET	<10ms	Fast	Kubernetes probe

/health/live	GET	<5ms	Fast	Kubernetes probe
/metrics	GET	<20ms	Fast	Prometheus scrape
/products	GET	<50ms	Fast	Normal DB query patterns
/products/:id	GET	<50ms	Fast	Single record lookup
/products	POST	<50ms	Fast	Simple DB write
/products/:id	DELETE	<50ms	Fast	Simple DB delete
/orders/:id	GET	<50ms	Fast	Indexed lookup
/notifications	GET	<50ms	Fast	Simple list query
/users	GET	<50ms	Fast	Admin user list
/products/:id	PUT	200-500ms	Medium	Update + inventory validation
/orders	POST	200-500ms	Medium	Multi-step transactional flow
/orders	GET	200-500ms	Medium	Filtered/sorted query
/inventory/sync	POST	300ms-2s	Slow	External service call
/products/bulk-import	POST	1-5s	Slow	Batch processing
/reports	POST	2-5s	Slow	Heavy aggregation queries
/payments/process	POST	1-3s (async)	Error-prone	20% failure, 10% timeout

3.5 Simulated Failure Scenarios

Scenario 1: Payment Gateway Timeout

- **Trigger:** 10% of payment calls (configurable via `/chaos/config`)
- **Behavior:** External call takes >5s, then times out
- **Expected Observability:**
 - ERROR log with timeout details and correlation ID
 - Span with `error: true` and `error.message: "Payment gateway timeout"`
 - `orderflow_payments_processed_total{status="timeout"}` counter increment
 - `orderflow_payment_gateway_duration_seconds` histogram shows 5s+ bucket

Scenario 2: Payment Gateway Error

- **Trigger:** 10% of payment calls return HTTP 500
- **Behavior:** External call returns immediately with error response
- **Expected Observability:**
 - ERROR log with gateway error response body
 - Span with `error: true` and gateway error code
 - Retry spans visible in trace
 - `orderflow_payments_processed_total{status="error"}` counter

Scenario 3: Inventory Service Degradation

- **Trigger:** Random additional 500ms-2s latency on warehouse API calls
- **Behavior:** Inventory sync takes longer than usual
- **Expected Observability:**
 - WARN log when response time exceeds threshold
 - Extended span duration visible in Tempo
 - `http_request_duration_seconds` histogram shift for inventory endpoints

Scenario 4: Database Slow Query

- **Trigger:** Report generation queries intentionally use non-optimal patterns
- **Behavior:** Report queries take 2-5s
- **Expected Observability:**
 - WARN log for any query >100ms
 - Multiple long DB spans in trace
 - `orderflow_db_query_duration_seconds` histogram shows high values

Scenario 5: Memory Pressure

- **Trigger:** Bulk import endpoint processes large payloads (1000+ products)
- **Behavior:** High memory usage during processing
- **Expected Observability:**
 - `process_resident_memory_bytes` gauge spike
 - Longer GC pauses visible in `nodejs_eventloop_lag_seconds`
 - Extended span duration

Scenario 6: Queue Backup

- **Trigger:** Slow consumer enabled via chaos config
- **Behavior:** Queue depth increases as consumers process slowly
- **Expected Observability:**
 - `orderflow_queue_depth` gauge increases
 - `orderflow_queue_processing_duration_seconds` histogram shifts
 - WARN log when queue depth exceeds threshold

Part 4: Data Model

Core Entities

User

Field	Type	Constraints	Notes
id	UUID	PK, auto-generated	
email	string	Unique, not null	
password	string	Not null, hashed	bcrypt hashed
name	string	Not null	
phone	string	Nullable	

address	string	Nullable	
role	enum	"customer" "admin"	Default: "customer"
status	enum	"active" "inactive"	Default: "active"
lastLoginAt	timestamp	Nullable	
createdAt	timestamp	Auto	
updatedAt	timestamp	Auto	

Product

Field	Type	Constraints	Notes
id	UUID	PK, auto-generated	
name	string	Not null	Indexed for search
description	text	Nullable	
price	decimal(10,2)	Not null	
category	string	Not null	Indexed
sku	string	Unique, not null	
stockQuantity	integer	Not null, default 0	
imageUrl	string	Nullable	
isActive	boolean	Default true	
createdAt	timestamp	Auto	
updatedAt	timestamp	Auto	

Order

Field	Type	Constraints	Notes
id	UUID	PK, auto-generated	
trackingId	string	Unique, auto-generated	Human-readable: ORD-XXXXXX
userId	UUID	FK → User	Indexed
status	enum	See status enum	Default: "pending"
totalAmount	decimal(10,2)	Not null	
taxAmount	decimal(10,2)	Not null	
shippingAddress	string	Nullable	
paymentStatus	enum	See payment status enum	Default: "pending"

paymentAttempts	integer	Default 0	
correlationId	UUID	Not null	For distributed tracing
createdAt	timestamp	Auto	Indexed
updatedAt	timestamp	Auto	

Order Status Enum: pending → confirmed → processing → shipped → delivered | cancelled
Payment Status Enum: pending → processing → paid | failed

OrderItem

Field	Type	Constraints	Notes
id	UUID	PK, auto-generated	
orderId	UUID	FK → Order	Indexed
productId	UUID	FK → Product	
quantity	integer	Not null, > 0	
unitPrice	decimal(10,2)	Not null	Price at time of order
totalPrice	decimal(10,2)	Not null	quantity * unitPrice

Payment

Field	Type	Constraints	Notes
id	UUID	PK, auto-generated	
orderId	UUID	FK → Order	Indexed
amount	decimal(10,2)	Not null	
status	enum	"pending" "success" "failed" "timeout"	
gatewayResponse	jsonb	Nullable	Simulated gateway response
attemptNumber	integer	Not null	
errorMessage	string	Nullable	
processingTime	integer	Nullable	milliseconds
correlationId	UUID	Not null	Links to order's correlation ID
createdAt	timestamp	Auto	

Notification

Field	Type	Constraints	Notes
id	UUID	PK, auto-generated	
userId	UUID	FK → User	Indexed
type	enum	"order_placed" "payment_success" "payment_failed" "order_shipped" "inventory_alert"	
title	string	Not null	
message	text	Not null	
isRead	boolean	Default false	
metadata	jsonb	Nullable	Related entity IDs
createdAt	timestamp	Auto	Indexed

InventoryLog

Field	Type	Constraints	Notes
id	UUID	PK, auto-generated	
productId	UUID	FK → Product	
previousQuantity	integer	Not null	
newQuantity	integer	Not null	
source	enum	"manual" "sync" "order" "bulk_import"	
discrepancy	integer	Nullable	Difference during sync
syncCorrelationId	UUID	Nullable	Links to sync operation
createdAt	timestamp	Auto	

Part 5: Database Seed Data

The database must be seeded with realistic data for meaningful observability testing:

Entity	Count	Notes
Users (Customer)	50	Varied registration dates
Users (Admin)	3	Pre-seeded
Products	150	Across 8-10 categories

Orders	1,500	Various statuses, date range spanning 6 months
OrderItems	4,000+	2-5 items per order
Payments	1,800	Includes retries, failures
Notifications	3,000+	Mixed read/unread
InventoryLogs	500	Sync history records

Additional Questions (Client Confirmation Required)

Required Clarifications

#	Question	Context
1	What PostgreSQL version should be targeted?	Affects available features like JSONB operators and query optimization hints
2	Should the queue system use Redis standalone or Redis Cluster?	Affects BullMQ configuration and connection setup
3	What is the desired data retention period for logs, metrics, and traces in the LGTM stack?	Affects storage configuration and cleanup policies
4	Should the API support rate limiting? If yes, what are the limits per endpoint category?	Could add another interesting observability dimension (429 responses)

Recommended Clarifications

#	Question	Context
1	Should the bulk import endpoint support streaming/chunked upload for very large files?	Affects memory usage patterns and observability signals
2	Should there be a WebSocket endpoint for real-time order status updates?	Would add interesting tracing scenarios for persistent connections
3	Is there a preference for the OpenTelemetry exporter protocol (gRPC vs HTTP)?	Affects OTel SDK configuration
4	Should the chaos endpoint support scheduled chaos (e.g., "inject failures for 5 minutes")?	Would add time-based failure testing capability

Feature Change Log

Version 1.0 (2026-03-02)

Change Type	Before	After	Source
Initial Creation	-	Full PRD	Client requirements file OrderFlow_260302_120000.md

Change Details

Initial PRD Creation

- **Source Document:** OrderFlow_260302_120000.md
- **Change Description:** Initial PRD generated from client requirements. All features, observability requirements, and API specifications derived from the input file.

Validation Results

Verification Checklist

- ☒ All user types from input are included (Customer, Admin)
- ☒ Each user type's permissions accurately reflected
- ☒ User relationships correct (Customer → Orders 1:N, Orders → Products N:M)
- ☒ All main features included (Product Catalog, Order Placement, Payment Processing, Report Generation, Inventory Sync, Health & Metrics)
- ☒ All module flows match input file
- ☒ Authentication method correct (JWT, email/password)
- ☒ Social login: N/A (not specified in input)
- ☒ All 3rd party integrations included (Simulated Payment Gateway, Simulated Warehouse API)
- ☒ Backend tech stack correct (NestJS)
- ☒ All domain terminology included (10 terms from input + 5 added for completeness)
- ☒ Chaos engineering endpoint included per input requirements
- ☒ Database seed data requirements included (100+ products, 1000+ orders)
- ☒ All observability requirements (Logging/Monitoring/Tracing) fully specified
- ☒ API response time categories match input (Fast/Medium/Slow/Error-prone/Async)
- ☒ All 6 simulated failure scenarios included

Modified Items

- Added 5 additional terminology entries (LGTM Stack, RED Metrics, OpenTelemetry, W3C Trace Context, Chaos Endpoint) for completeness
- Expanded API endpoints beyond input to include standard CRUD operations implied by features
- Added Data Model section (implied by features but not explicitly in input)
- Added Database Seed Data section (specified as requirement in input's Additional Information)

Validation Complete

- Total items verified: 16
- Items modified: 0
- Items expanded with implied details: 4
- Items moved to questions: 4 required + 4 recommended